

EventFlow 2026

Sistema de Gestión de Eventos
Informe Técnico Completo

Fecha: 27 de May de 2026
Stack: MongoDB · Express.js · Node.js · Vanilla JS
Despliegue: Vercel · MongoDB Atlas

Generado automáticamente por análisis estático del repositorio

1. Resumen Ejecutivo

EventFlow 2026 es una plataforma web full-stack para la gestión integral de eventos, reservas y análisis de datos. Implementada con el stack **MEN** (MongoDB, Express.js, Node.js) en el backend y una **SPA en JavaScript vanilla** en el frontend, la aplicación está diseñada para desplegarse en entornos **serverless** mediante Vercel, con base de datos en MongoDB Atlas.

El sistema gestiona tres roles de usuario diferenciados (administrador, organizador y cliente), ofrece un dashboard analítico con 7 pipelines de agregación MongoDB, y soporta el ciclo de vida completo de un evento: creación, reserva, conclusión y reseña.

Dimensión	Valor
Líneas de código totales	~5 000+
Lenguaje principal	JavaScript (Node.js 24.x)
Base de datos	MongoDB (Mongoose 8.5)
Framework backend	Express.js 4.19
Frontend	Vanilla JS ES6 · Tailwind CSS · Chart.js
Despliegue	Vercel (serverless) · MongoDB Atlas
Colecciones BD	users, events, reservations, reviews
Endpoints REST	26 endpoints en 6 routers
Aggregation pipelines	7 pipelines analíticos
Roles de usuario	admin · organizador · cliente

2. Arquitectura General

La aplicación sigue un patrón **MVC con separación por capas**. El frontend actúa como cliente SPA que consume la API REST; el backend expone los endpoints y aplica la lógica de negocio; y MongoDB almacena todos los datos.

2.1 Diagrama de capas



```

■ Mongoose ODM
■
■ MONGODB (Atlas o Local) ■
■ Collections: users · events ·
■ reservations · reviews ■

```

Ruta	Descripción
config/db.js	Conexión MongoDB con connection caching
models/	4 schemas Mongoose (User, Event, Reservation, Review)
controllers/	6 controladores con lógica de negocio
routes/	6 archivos de rutas Express
public/	Frontend SPA (HTML + JS vanilla)
scripts/seed.js	Generador de datos de prueba (1 000+ docs)
api/index.js	Entry point serverless para Vercel
server.js	Aplicación Express principal (106 líneas)

Paquete	Versión	Propósito
Node.js	24.x	Runtime JavaScript del lado servidor
Express.js	^4.19.2	Framework web para la API REST
Mongoose	^8.5.1	ODM (Object Document Mapper) para MongoDB
dotenv	^16.4.5	Gestión de variables de entorno
cors	^2.8.5	Middleware CORS configurable
nodemon	^3.1.4	Hot-reload en desarrollo (devDependency)

3.2 Frontend (CDN, sin bundler)

Librería	Propósito
Tailwind CSS	Framework CSS utilitario para estilos responsivos
Chart.js	Gráficas interactivas (línea, donut, barras)
Google Fonts	Syne (display) · JetBrains Mono (datos) · DM Sans (body)
Vanilla JS ES6	Lógica SPA sin framework adicional

3.3 Infraestructura

Servicio	Rol
Vercel	Hosting serverless (frontend + API functions)
MongoDB Atlas	Base de datos en la nube (cluster compartido)
GitHub	Control de versiones y CI/CD implícito

4. Modelos de Datos (MongoDB / Mongoose)

Las cuatro colecciones se interrelacionan mediante referencias ObjectId y se acceden con **populate()** y **populate anidado** en las consultas que requieren datos de múltiples colecciones.

4.1 User

nombre	String · 2-80 chars · requerido
correo	String · único · lowercase · validación regex email
password	String · 4+ chars · select:false en queries
rol	Enum: 'admin' 'organizador' 'cliente' (default: 'cliente')
avatar	String URL · opcional
Índices	{ rol: 1 }

4.2 Event

titulo	String · 4-120 chars · requerido
descripcion	String · 0-1 000 chars
fecha	Date · requerida

lugar	String · requerido
categoria	Enum: Música Tecnología Arte Deportes Educación Negocios Otro
precio	Number · ≥ 0 · requerido
capacidad	Number · min:1 · default:100
organizadorId	ObjectId $\rightarrow$ User · requerido
activo	Boolean · default:true (borrado lógico)
Virtual	finalizado: fecha < Date.now()
Índices	{ fecha:1 } { categoria:1 } { organizadorId:1 } { activo:1 }

4.3 Reservation

usuarioid	ObjectId $\rightarrow$ User · requerido
eventoid	ObjectId $\rightarrow$ Event · requerido
cantidad	Number · 1-10 · requerido
estado	Enum: 'confirmada' 'cancelada' 'pendiente' (default: 'confirmada')
fechaReserva	Date · default: Date.now()
precioUnitario	Number · snapshot del precio al momento de reservar
notas	String · 0-300 chars
Virtual	totalReserva: precioUnitario $\times$ cantidad
Índices	{ eventoid:1 } { usuarioid:1 } { estado:1 } { fechaReserva:1 } + compuesto parcial

4.4 Review

eventoid	ObjectId $\rightarrow$ Event · requerido
usuarioid	ObjectId $\rightarrow$ User · nullable
autorNombre	String · 2-80 chars · requerido
calificacion	Number · 1-5 · requerido
comentario	String · 10-500 chars · requerido
Índices	{ eventoid:1, createdAt:-1 }

4.5 Relaciones entre modelos

```
User (organizador) 1:N Event
User (cliente) 1:N Reservation
Event 1:N Reservation
Event 1:N Review
User 0:N Review (nullable)
```

5. API REST – Endpoints

5.1 Autenticación /api/auth

Método	Ruta	Descripción
POST	/login	Login con correo y contraseña. Devuelve {id, nombre, correo, rol}

5.2 Eventos /api/events

Método	Ruta	Descripción
GET	/	Listar eventos. Query: categoria, q, page, limit, past
GET	/:id	Obtener evento por ID (con populate del organizador)
POST	/	Crear evento. Body: titulo, fecha, lugar, categoria, precio, capacidad, organizadorId
PUT	/:id	Actualizar campos del evento
DELETE	/:id	Borrado lógico (activo = false)

5.3 Reservas /api/reservations

Método	Ruta	Descripción
GET	/	Listar con populate anidado. Query: estado, usuarioId, eventId, page, limit
GET	/:id	Obtener reserva individual con datos de usuario y evento
POST	/	Crear reserva. precioUnitario capturado automáticamente del evento
PUT	/:id	Actualizar estado, cantidad o notas
DELETE	/:id	Cancelar reserva (estado → 'cancelada')

5.4 Usuarios /api/users

Método	Ruta	Descripción
GET	/	Listar usuarios. Query: rol, q (búsqueda)
GET	/:id	Obtener usuario por ID
POST	/	Crear usuario
PUT	/:id	Actualizar perfil
DELETE	/:id	Eliminar usuario

5.5 Reseñas /api/reviews

Método	Ruta	Descripción
GET	/stats	Promedio y conteo de reseñas por evento
GET	/:eventoid	Reseñas de un evento (promedio + total)
POST	/	Crear reseña (solo para eventos ya concluidos)
DELETE	/:id	Eliminar reseña

5.6 Dashboard /api/dashboard

Método	Ruta	Descripción
GET	/kpis	KPIs globales: ingresos, reservas, cancelaciones, calificación...
GET	/top-eventos	Top 5 eventos con más reservas e ingresos generados
GET	/ingresos-por-categoria	Desglose de ingresos y tickets por categoría
GET	/ranking-organizadores	Ranking de organizadores por ingresos totales
GET	/historico-mensual	Serie temporal mensual. Query: ?year=YYYY
GET	/top-calificados	Top 5 eventos con mejor calificación promedio
GET	/reservas-por-estado	Distribución de reservas por estado

5.7 Health Check /api/health

GET /api/health	Retorna: ok, status, environment, database (connected/disconnected), timestamp ISO 8601
-----------------	---

6. Aggregation Pipelines – Analytics

El módulo **statsController.js** concentra 7 aggregation pipelines de MongoDB que alimentan el dashboard analítico. Todas usan **\$lookup** para joins, **\$group** para agregados y **\$project** para dar forma al resultado.

Pipeline	Colección base	Operadores clave	Salida principal
getTopEventos	reservations	\$group · \$sort · \$limit · \$lookup	Top 5 por entradas e ingresos
getIngresosPorCategoría	reservations	\$addFields · \$lookup · \$group	Ingresos por categoría + ticket promedio
getRankingOrganizadores	reservations	doble \$lookup (events + users)	Ranking por ingresos totales
getHistoricoMensual	reservations	\$year/\$month · \$group · \$sort	Serie temporal YYYY-MM
getKPIs	múltiples	5 aggregates paralelos	8 KPIs globales
getTopCalificados	reviews	\$group · \$sort · \$limit · \$lookup	Top 5 por calificación promedio
getReservasPorEstado	reservations	\$group por estado	Distribución confirmada/cancelada/pendiente

6.1 Ejemplo – getKPIs

Este pipeline ejecuta **5 aggregates en paralelo** (Promise.all) sobre las colecciones reservations y reviews para calcular:

- totalReservas, totalEntradas, ingresoTotal (con filtro opcional por año/mes)
- tasaCancelacion: canceladas ÷ totales × 100
- totalEventos y proximosEventos (fecha ≥ hoy)
- promedioCalificacion y totalReseñas desde la colección reviews

7. Frontend – Single Page Application

El frontend es una SPA implementada sin frameworks adicionales. Dos archivos conforman toda la capa de presentación: **index.html** (markup y configuración de estilos) y **app.js** (lógica de navegación, llamadas a la API y renderizado dinámico).

Archivo	Líneas	Responsabilidad
public/index.html	1 438	Markup, Tailwind config CDN, Chart.js, Google Fonts, 5 views
public/app.js	1 881	Estado global, fetch API, renderizado DOM, gráficas, localStorage

7.1 Vistas disponibles

Vista	Acceso	Contenido
Login overlay	Todos	Formulario correo/password · Botones demo por rol
Explorador de eventos	Todos	Grid responsivo · Filtros · Paginación · Modal reservar
Eventos concluidos	Todos	Listado pasados · Formulario reseña · Calificaciones ■
Dashboard analítico	Solo admin	5 KPI cards · Gráfica línea dual · Donut · Barras · Tabla
Admin panel	Solo admin	CRUD completo: Eventos · Usuarios · Reservas (tabs)

7.2 Características del frontend

- Dark/light mode toggle mediante clase CSS
- Diseño responsivo: mobile, tablet y desktop
- Sistema de toast notifications (success/error)
- Sistema de modales genérico
- Formato de moneda y fechas en locale es-CO
- Emoji mapping por categoría de evento
- Color mapping por estado de reserva
- Fallback demo si el backend no está disponible
- Persistencia de sesión en localStorage

8. Configuración y Despliegue

8.1 Variables de entorno (.env)

Variable	Ejemplo	Descripción
PORT	3000	Puerto del servidor Express
MONGO_URI	mongodb+srv://user:pass@cluster.net/db	Cadena de conexión MongoDB
NODE_ENV	development production	Entorno de ejecución
CORS_ORIGIN	(vacío)	Origen CORS personalizado (opcional)

8.2 Scripts npm

Script	Comando	Uso
start	node server.js	Producción

Script	Comando	Uso
dev	nodemon server.js	Desarrollo con hot-reload
seed	node scripts/seed.js	Poblar BD con 1 000+ documentos de prueba

8.3 Connection caching (serverless)

config/db.js implementa un patrón de connection caching optimizado para entornos serverless: reutiliza la conexión existente si `mongoose.connection.readyState === 1`, evitando abrir una nueva conexión en cada invocación de función serverless. Parámetros clave:

maxPoolSize	5
minPoolSize	1
serverSelectionTimeoutMS	5 000 ms
socketTimeoutMS	10 000 ms
Detección serverless	Variable de entorno VERCEL

8.4 Vercel (vercel.json)

Todos los path **/api/*** son reescritos a la función serverless **/api/index.js**, que importa y exporta la aplicación Express. El frontend estático se sirve desde **/public**.

9. Consideraciones de Seguridad

9.1 Puntos débiles identificados

- **Contraseñas en texto plano:** authController.js compara password directamente sin hashing (bcrypt). Crítico para producción.
- **Sin JWT / sesiones:** la autenticación se almacena solo en localStorage del cliente, sin token firmado ni expiración.
- **Sin middleware de autorización:** todos los endpoints son accesibles sin validación de rol en el servidor.
- **CORS abierto por defecto:** si CORS_ORIGIN está vacío, el middleware puede permitir cualquier origen.
- **Sin rate limiting:** no hay protección contra abuso de los endpoints.
- **Sin validación de entrada en servidor:** más allá de los schemas Mongoose, no hay sanitización explícita.

9.2 Buenas prácticas presentes

- Campo password con select:false en queries (no se expone en listados de usuarios).
- .gitignore incluye .env y node_modules.
- Borrado lógico de eventos (activo:false) en lugar de eliminación física.
- Snapshot de precio en reserva (precioUnitario) evita manipulación posterior.
- Health check endpoint para monitoreo de disponibilidad.
- Error handler global en Express captura errores no controlados.

10. Testing

El proyecto incluye un script de test en bash (**test-checkpoint-5.sh**) que verifica el correcto funcionamiento del health check y el manejo de errores. No existe suite de tests unitarios ni de integración automatizados.

Test	Qué verifica
Health Check endpoint	Campos: ok, status, environment, database, timestamp
Database status	Estado connected/disconnected de la BD
Environment detection	Valor correcto de NODE_ENV
Timestamp format	Formato ISO 8601 en la respuesta
Error handling	Respuesta con ok:false y campo message

Recomendaciones de testing

- Añadir suite Jest o Mocha para tests unitarios de controladores.
- Implementar tests de integración con Supertest para todos los endpoints.
- Usar MongoDB Memory Server para tests sin dependencia de Atlas.
- Agregar tests E2E con Playwright o Cypress para el frontend.

11. Métricas del Proyecto

Archivo / Módulo	Líneas	Descripción
public/app.js	1 881	Lógica frontend completa
public/index.html	1 438	Markup + estilos inline
controllers/statsController.js	~400	7 aggregation pipelines
controllers/reservationsController	121	CRUD reservas
controllers/eventsController.js	100	CRUD eventos

Archivo / Módulo	Líneas	Descripción
server.js	106	App Express principal
models/ (4 archivos)	~400	Schemas Mongoose
routes/ (6 archivos)	~200	Definición de rutas
scripts/seed.js	~150	Generador de datos
TOTAL estimado	~5 000+	Codebase completo

Métrica	Valor
Colecciones MongoDB	4 (users, events, reservations, reviews)
Índices definidos	11 índices simples + 1 compuesto parcial
Endpoints REST	26 endpoints en 6 routers
Aggregation pipelines	7 pipelines analíticos
Roles de usuario	3 (admin, organizador, cliente)
Documentos seed	1 000+ documentos de prueba
Categorías de eventos	7 (Música, Tecnología, Arte, Deportes, Educación, Negocios, Otro)
Estados de reserva	3 (confirmada, cancelada, pendiente)

12. Conclusiones y Recomendaciones

EventFlow 2026 es un proyecto **completo y funcional** que demuestra un dominio sólido del stack MEN y de las capacidades analíticas de MongoDB. La arquitectura serverless-ready y el uso de connection caching hacen que la aplicación sea apta para despliegues en la nube sin configuración adicional.

Fortalezas

- Aggregation pipelines bien diseñados con joins, series temporales y rankings.
- Modelo de datos normalizado con índices estratégicos para performance.
- SPA funcional sin dependencias de frameworks pesados.
- Arquitectura serverless-ready con connection caching.
- Seed script completo para desarrollo y testing.
- Borrado lógico y snapshot de precios como buenas prácticas de datos.

Áreas de mejora prioritarias

Prioridad	Área	Recomendación
ALTA	Seguridad	Implementar hashing de contraseñas con bcrypt
ALTA	Autenticación	Adoptar JWT con expiración y refresh tokens
ALTA	Autorización	Añadir middleware de roles por endpoint
MEDIA	Testing	Suite Jest + Supertest para todos los controladores
MEDIA	Validación	Sanitización de inputs con express-validator o Zod
MEDIA	Rate limiting	helmet + express-rate-limit para protección básica
BAJA	Frontend	Migrar a componentes web o framework ligero (Preact/Svelte)
BAJA	Observabilidad	Añadir logging estructurado (Winston/Pino) y tracing